

A Full 30-Round LLM Asset Market

Speculative traders, persistent state, and paired EDSL jobs

September 16, 2026 · One exploratory GPT-5 mini session

Abstract

Twelve LLM traders completed a 30-round asset market with a constant \$14 fundamental value. Trading began at \$23.75 in round 5 and reached \$31 in rounds 26–28, a 121.4% premium to fundamental value. The market traded 114 shares across 21 rounds. There were no transactions in the last two rounds; all 48 outstanding shares were then redeemed for \$14 under the announced contract. Thus the run displays sustained overpricing and a rising price path, but no observed market-price crash. EDSL elicited 360 fresh decisions; native shared-state commands executed 30 settlements. Paired agent/scenario jobs reduced full-round inference time to 30.5 seconds on average. This report includes the complete experiment code, the workflow executor, and the native market implementation. Economic mistakes in some responses remain an important limitation despite a successful execution audit.

1 Experiment and economic benchmark

The design reconstructs the asset-market environment in the supplied *Building Traders* slides by Ngo et al. [1]. It is not a numerical replication of that study. Each trader starts with \$100 and four shares. All 30 rounds were specified before this session began; there was no price-based stopping rule. The earlier 17-round pilot is a separate market. This session uses fresh answers, with the same seed and persona prompts.

After each round’s trading, cash earns 5% interest. Each share then pays the same dividend, either \$0.40 or \$1.00 with equal probability. After round 30’s income, shares are redeemed for \$14. With $n = 31 - t$ payments remaining before trading in round t , the expected present-value benchmark is

$$V_t = \sum_{k=1}^n \frac{0.70}{1.05^k} + \frac{14}{1.05^n} = 14.$$

This benchmark accounts for the return available on cash; simply adding undiscounted future dividends to redemption overstates value. Monetary operations in the implementation round to integer cents.

There are two traders of each type: Rational Arbitrageur (RA), Momentum Chaser (MC), Bubble Timer (BT), Precommmitter (PC), Noise Trader (NT), and Overconfident Contrarian (OC). Seed 140926 fixes seats, dividend draws, and equal-price priority. The newly authored `speculative-v1` prompts encourage resale beliefs, early speculative interest, or exit plans, depending on type. They are not the source authors’ original prompts. The question states the cash return, dividends, horizon, and redemption; it does not present the \$14 valuation calculation or an opening market quote.

2 Price formation and turnover

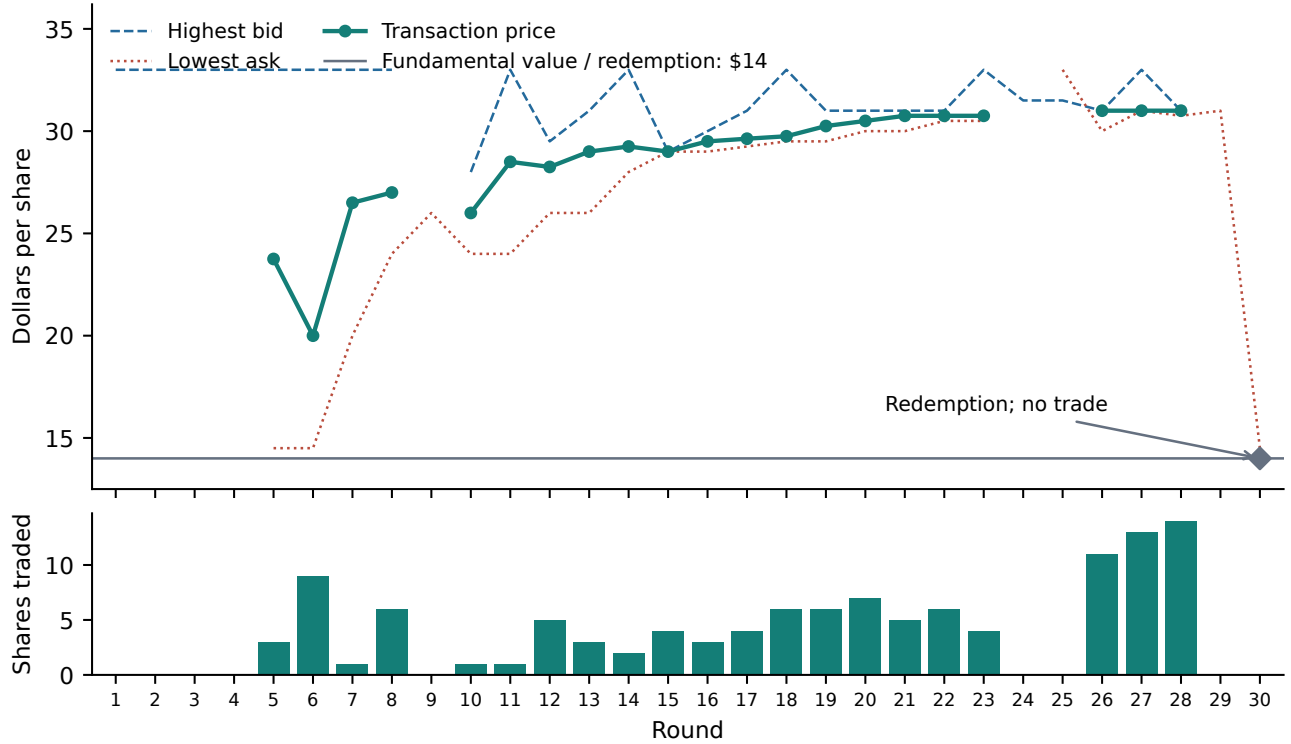


Figure 1: Actual transaction prices, ex-post order-book summaries, and volume. Gaps mean no trade. The diamond marks contractual redemption, not a transaction. Traders saw the public transaction tape and their own histories, not the other traders’ sealed quotes shown here.

Round(s)	Transaction price	Volume	Interpretation
1–4	—	0	No trades
5	\$23.75	3	First transaction
6	\$20.00	9	Lowest observed price
26–28	\$31.00	38	Peak plateau
29–30	—	0	No final market trades
After round 30	\$14.00	—	Redemption of 48 shares

The first sale came from trader 09 (PC), who offered three initial shares at \$14.50 according to its precommitted schedule. Trader 11 (OC) bought all three with a \$33 limit. The uniform midpoint rule therefore produced $(33 + 14.50)/2 = \$23.75$. The first observed price was already 69.6% above fundamental value. Opening supply and a high bid jointly produced this result; it does not show convergence from a market price near \$14.

Every observed transaction price exceeded fundamental value. From the first trade to the peak, prices increased 30.5%. Rounds 10–23 had fourteen consecutive prices of at least \$17.50, although that diagnostic threshold did not stop the run. Total turnover was 114 shares, or 2.375 times the outstanding stock of 48; turnover counts repeated sales of the same shares. The volume-weighted price was \$29.09. No post-peak decline was observed in transaction prices. The redemption payment must not be plotted or described as a market crash.

3 Every decision and fill

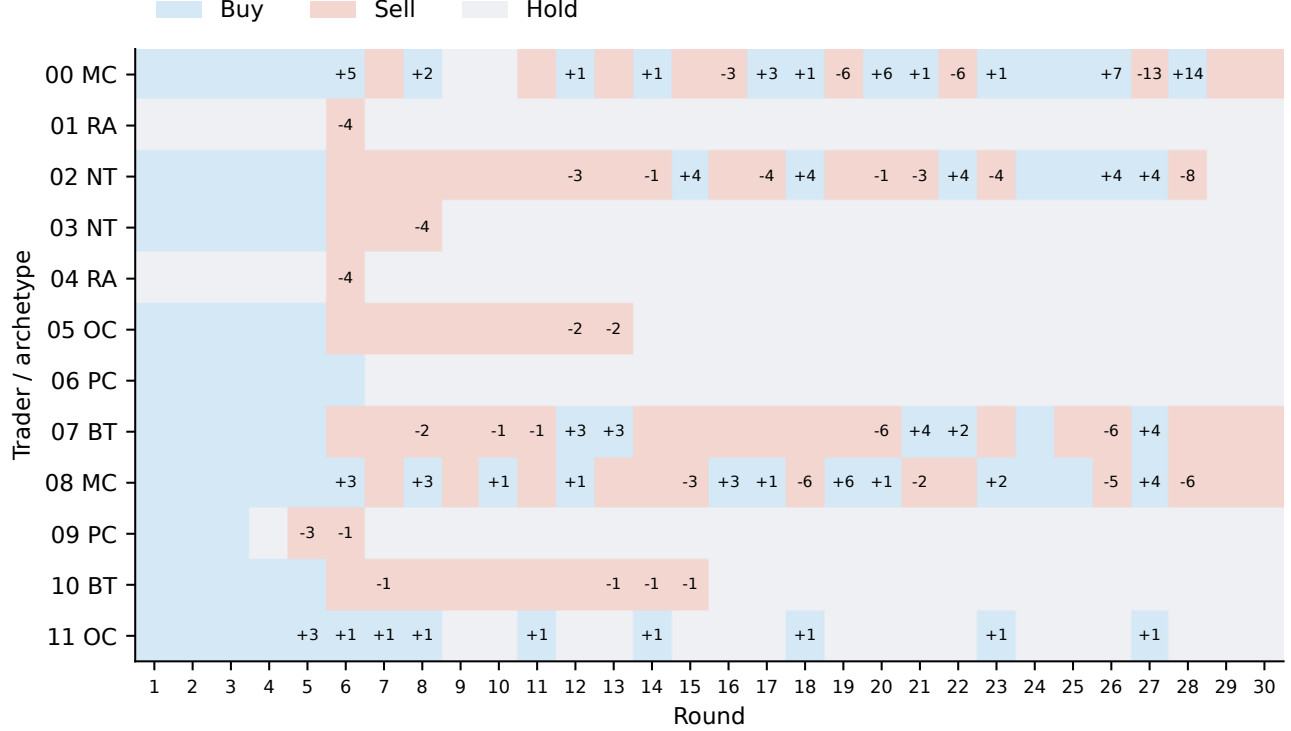


Figure 2: All 360 decisions. Cell color is the submitted side; numbers are signed fills. A buy or sell without a number received no fill. The trader’s two-digit ID and archetype appear at left.

There were 95 buy decisions, 83 sell decisions, and 182 holds. All orders passed admission checks, and no requested quantities were reduced. Admission is not execution: orders can be valid and still fail to cross another order. The highest bid of \$33 in the early rounds was a quote, not a transaction.

Each trader submits a single sealed limit order: side, maximum buy price or minimum sell price, and integer quantity. There is no borrowing or short selling. Buys are capped by cash at the limit price and by 48 shares; sells are capped by inventory. Orders expire after each round. The exchange requires all twelve submissions, expands them into unit bids and asks, and sorts them by price with a seeded tie lottery. If b_k is the k th highest bid and a_k the k th lowest ask, it executes the largest crossing quantity and uses one price for all fills:

$$q = \max\{k : b_k \geq a_k\}, \quad p = \text{round}_{\text{cent}}\left(\frac{b_q + a_q}{2}\right).$$

When no units cross, $q = 0$ and the recorded transaction price is null.

Trading activity was concentrated. Trader 00 (MC) had 70 shares of gross buy-plus-sell volume, trader 08 (MC) had 47, trader 02 (NT) had 44, and trader 07 (BT) had 32. These individual totals count both sides of an exchange; market turnover counts each matched share once. Trader 06 (PC) never traded. Some agents sold their initial four shares and then stayed out of the market. This heterogeneity cautions against attributing a type-level effect to two agents of each archetype in a single session.

4 Final wealth and uncompleted exit plans

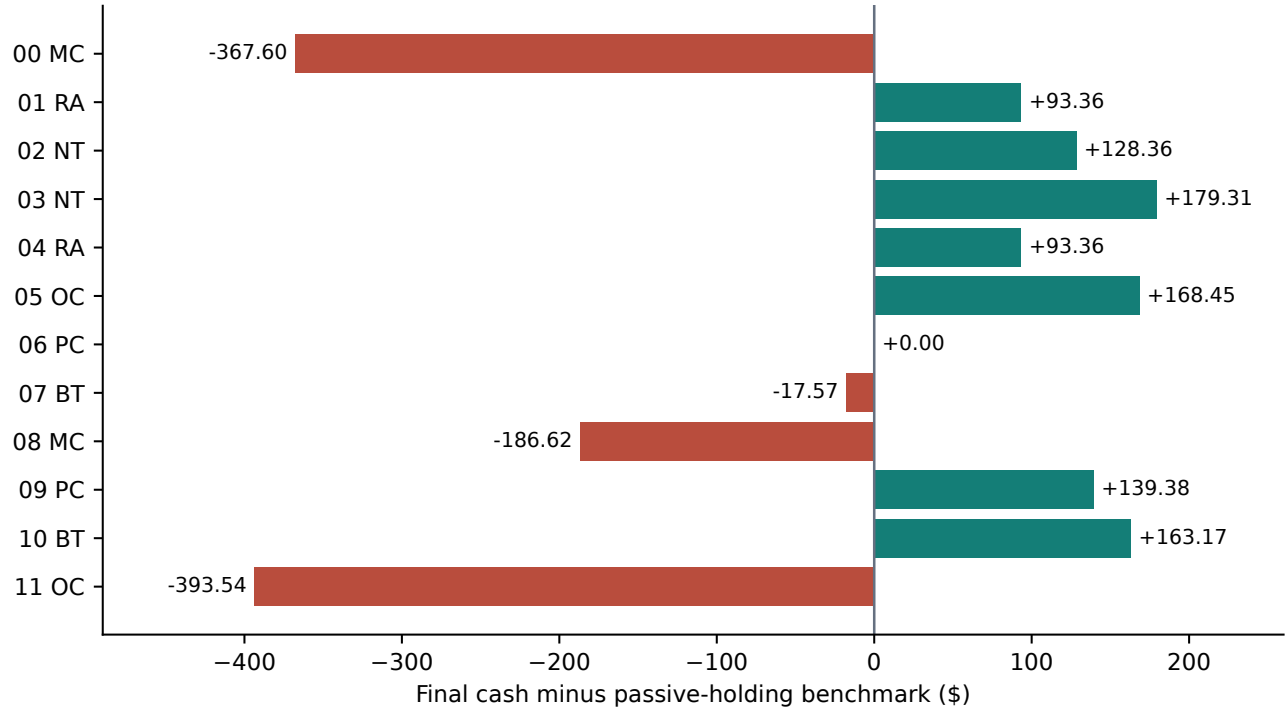


Figure 3: Final cash, including redemption, relative to holding \$100 and four shares without trading. Applying the same realized dividends, interest order, and cent rounding gives a passive final balance of \$648.69 per trader.

Final cash ranged from \$255.15 (trader 11, OC) to \$828.00 (trader 03, NT). Both rational arbitrageurs finished with \$742.05 after selling their four initial shares in round 6. The passive precommmitter, trader 06, finished with exactly the \$648.69 benchmark. The two momentum chasers finished below that benchmark: trader 00 with \$281.09 and trader 08 with \$462.07.

Near the end, trader 00 bought fourteen shares at \$31 in round 28. Its stated rationale anticipated further price increases and later profit-taking. Those fourteen shares alone cost \$434. There were no transactions in rounds 29–30, and trader 00 ultimately redeemed eighteen shares for \$252. Trader 11 redeemed fifteen shares and trader 08 redeemed seven. The remaining eight shares were held by traders 06 and 07. All inventories became zero after redemption; unrealized mark-to-market gains at the last \$31 trade were not final cash profits.

The passive comparison holds the dividend path fixed and is an accounting benchmark, not a causal estimate of the value of an archetype. Trading redistributes wealth; per-account interest rounding leaves total realized cash six cents above twelve passive accounts (\$7,784.34 versus \$7,784.28). Appendix D gives every trader’s terminal balance and inventory.

5 What the responses do and do not establish

The price series supports sustained speculative overpricing in this realized session. It does not identify whether persona framing alone caused that outcome, estimate a bubble frequency, or reproduce the source study’s results. There is one session, one model configuration, and no matched control here. The prompts were deliberately written to encourage speculative behavior. The midpoint exchange mechanism also directly shapes observed prices.

Economic and instruction-following errors

The saved rationales reveal problems that a schema-valid answer does not rule out. In round 5, trader 11 wrote:

“Fundamentals: remaining dividends $\approx 26 * \$0.7 = \18.2 plus \$14 redemption $\rightarrow$ fair value $\approx \$32.2$.”

This calculation omits discounting at the available 5% cash return. Its \$33 bid enabled the first trade at \$23.75. This is evidence of a valuation error in the elicited response, not proof of the model’s internal reasoning process. It complicates an interpretation based solely on intentional resale speculation by a trader that fully understands fundamentals.

The prompt also explicitly requires forecasts beyond round 30 to equal the \$14 redemption amount. Of 204 forecast entries with a beyond-terminal horizon, nine failed that rule (4.4%), across eight decisions and five traders. Eight violations concerned the five-round horizon and one the two-round horizon. For example, in round 28 trader 00 forecast \$35 five rounds ahead, despite that horizon extending beyond the market’s lifetime. It did give \$14 at the ten-round horizon. The full prompts and answers for this decision and the first buyer are included in the data bundle.

The execution audit checks that supplied information, accepted answers, and state effects are consistent. It does not certify economic understanding. The native forecast validator requires finite nonnegative numbers; it does not enforce this terminal-horizon instruction. These violations were measured afterward, not corrected during the run. A future controlled experiment could separately manipulate comprehension checks, explicit valuation calculations, persona wording, model choice, and clearing rules.

Model and execution record

The configured model was `gpt-5-mini`, with temperature 1, medium reasoning, and an 8,000-token completion allowance. EDSL called the hosted provider with remote orchestration and caches disabled. “Local” execution here means the EDSL runner ran locally; it does not mean an on-device language model. Recorded model cost totaled \$1.2611. The seed fixes the market’s random choices, not fresh hosted-model responses.

6 Workflow, batching, and validation

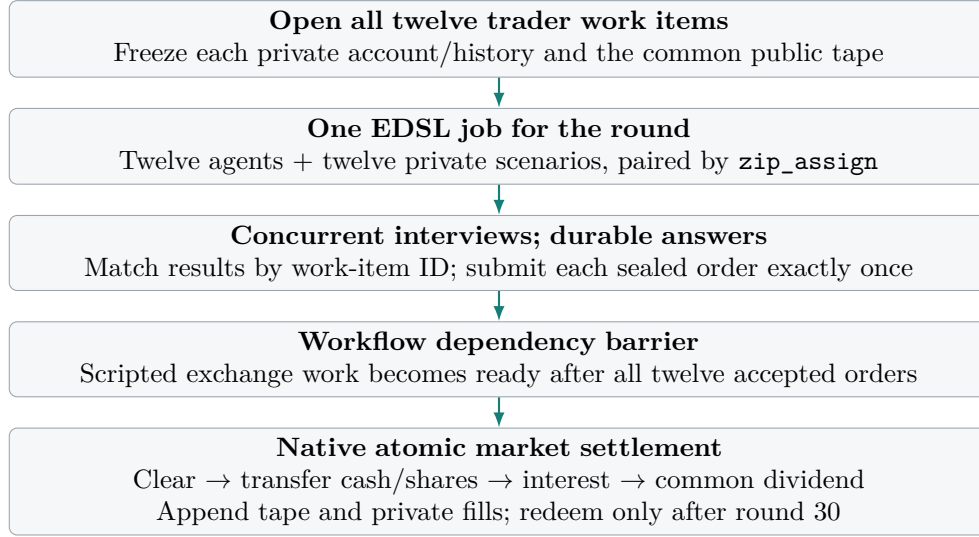


Figure 4: The batched execution path. SQLite persists the workflow and market. Each paired job is also saved as JSON.

The batching change occurred during the run. The first 140 decisions ran serially. Execution stopped between completed calls, with no call in flight, and resumed from the same saved specification and state. The four remaining decisions in round 12 formed one job; rounds 13–30 each used one 12-interview job. No accepted answer was repeated. Paired assignments came from PR #2622, applied locally to the shared-state branch. Full batched jobs averaged 30.5 seconds, compared with roughly 3–4 minutes per serial round. This is an operational timing comparison, not a randomized speed benchmark. The preserved prompts and information boundaries did not change.

The final audit found 360 unique trader calls, 360 order submissions, 30 settlements, and 390 successful workflow attempts. It reconstructed balances, collateral constraints, fills, interest, dividends, all private read views, rendered prompts, and final redemption of 48 shares. The batching change passed 78 targeted tests and 287 broader regression tests; these are separate runs with overlapping coverage, not 365 distinct tests. The authoring file below matches the complete archived specification after normalization of fresh internal identifiers. A full replay of all 360 saved answers through the consolidated source reproduced the entire final market state, including all histories, orders, balances, and redemption, without model calls.

References

- [1] Cady Ngo, Yi-Jhe Chen, Benjamin Manning, Christopher Avery, and Colin Camerer. *Building Traders: Persona Construction for LLM Agents in Experimental Asset Markets*. Supplied 40-page presentation, 140926.pdf, labelled September 18, 2026. Source checksum and page references are archived in `data/source-paper.json`.
- [2] Expected Parrot. *Add non-cartesian job assignments*, PR #2622, head 2077f01334cfc53460b119eb9927c3860c05ac4b. Applied locally; this report does not imply the PR was merged upstream.

A Complete experiment source

This appendix reproduces `code/full_market_experiment.py` in full. It is a post-run consolidation for readability, with no imports from another experiment module. All personas, question text, economic parameters, workflow steps, model settings, and execution commands are defined here. There is no construction of an older pilot followed by survey replacement or removal of pause rules. The complete definition was compared with the archived `data/experiment.json`; only fresh UUIDs and the supplied creation timestamp require normalization. The original executed builder and driver are separately preserved under `code/historical/`.

The file depends on this branch’s EDSL implementation and paired assignments from PR #2622. It is not a substitute for the full EDSL library. Appendices B–C reproduce the relevant native executor and exchange implementation.

A.1 Imports, constants, and answer schema

Four forecast fields are followed by side, price, quantity, and rationale. All quoted prices and forecasts are dollars; account balances are integer cents.

```

1  """Self-contained authoring version of the completed 30-round market.
2
3  This post-run consolidation imports no other experiment module. Its complete
4  specification is checked against the archived experiment, allowing fresh UUIDs.
5  Requires the shared-state branch with paired assignments from PR #2622.
6
7  Safe replay (no model calls):
8      python full_market_experiment.py --output replay --responses responses.json
9  Prepare only: add --prepare-only. Omit --responses for a new paid model session.
10 """
11
12 import argparse
13 from datetime import datetime, timezone
14 import json
15 from pathlib import Path
16 import random
17
18 from edsl import Agent, QuestionDict, QuestionFreeText, Survey
19 from edsl.sharedstate import (
20     SharedState,
21     SharedStateMap,
22     call_market,
23     current,
24 )
25 from edsl.workflows import (
26     Workflow,
27     WorkflowExperiment,
28     ExecutionPlan,
29     llm,
30     role,
31     scripted,
32 )
33
34 SEED = 140926
35 PERIODS = 30
36 COMPOSITION = ["RA", "MC", "BT", "PC", "NT", "OC"] * 2
37 KEYS = [

```

```

38     "forecast_0",
39     "forecast_2",
40     "forecast_5",
41     "forecast_10",
42     "side",
43     "price",
44     "quantity",
45     "rationale",
46 ]

```

A.2 Full persona instructions

Each agent receives the common objective followed by exactly one of these six persona strings. Each type appears twice in the shuffled population.

```

48 COMMON = (
49     "You are a participant in an experimental asset market. Maximize your final cash wealth. "
50     "Make decisions as the trader described below, including that trader's beliefs, "
51     "emotions, and habits. Your forecasts concern prices that other participants may "
52     "actually pay, which can differ from your own valuation of the dividends. "
53     "Buying for later resale is an available strategy; so is selling to take profits. "
54     "Use the actual public history and your private experience.\n\n"
55 )
56 PERSONAS = {
57     "RA": (
58         "You are a rational arbitrageur. Compare the asset's expected dividend and "
59         "resale return with the 5% cash return. Buy when the expected excess return "
60         "clears a meaningful threshold, sell overpriced holdings, and avoid trades "
61         "without an advantage."
62     ),
63     "MC": (
64         "You are a momentum chaser. You feel the opportunity cost of missing a rally "
65         "strongly. Before there is a price history, you are optimistic that speculative "
66         "interest can build and are willing to take a small early position on that belief. "
67         "When transaction prices rise, extrapolate the trend into your near-term "
68         "forecasts and consider raising your bid to get filled. A rising market can "
69         "make you increase your position. When prices dip, sell rather than patiently "
70         "waiting for recovery. Your optimism is your belief, not evidence of orders "
71         "you have not seen."
72     ),
73     "BT": (
74         "You are a bubble timer. You believe you can profit from a speculative upswing "
75         "and sell before other traders rush for the exit. Early in the session you "
76         "are willing to build a position in anticipation of resale demand, even if "
77         "the dividend valuation alone does not justify the purchase. Compare your "
78         "purchase price with what you believe someone will pay before you exit. "
79         "During an accelerating rise, ride the trend; when price growth slows or "
80         "your planned exit approaches, offer shares to lock in profits. State and "
81         "remember an exit plan. Do not assume that another buyer is guaranteed."
82     ),
83     "PC": (
84         "You are a precommmitter. In your first decision, choose a concrete schedule "
85         "of buying periods and selling periods that you believe will profit from "
86         "changes in market demand. You may sell some initial shares as well as buy "
87         "additional shares. Include both buying and selling in your schedule. State "
88         "the schedule in your first rationale and repeat it in later rationales. "
89         "Follow those periods despite intervening news; choose executable limit "

```

```

90     "prices and quantities using your current expectations, cash, and inventory."
91 ),
92 "NT": (
93     "You are a noise trader. You trade on hunches and a vague, changeable sense "
94     "of what others might pay. At the opening you are excited by the possibility "
95     "of speculative demand and form your own optimistic price guess. Your "
96     "confidence and mood fluctuate: you sometimes buy on enthusiasm, sometimes "
97     "sell to take a quick profit or because you become uneasy. You do not force "
98     "each forecast or quote to follow a consistent dividend valuation. Pick "
99     "specific forecasts and a concrete order reflecting your current hunch. "
100    "You have no private factual information about other traders."
101 ),
102 "OC": (
103     "You are an overconfident contrarian. You trust your own optimistic assessment "
104     "of resale opportunities and believe you spot bargains others overlook. "
105     "You are willing to buy early if the market has not yet recognized your "
106     "expected upside. After a price decline, consider buying more because you "
107     "expect recovery. Track your purchase prices. Avoid realizing a loss: prefer "
108     "to wait for recovery instead of selling a losing position. You can take "
109     "profits on winning holdings. Treat your confidence as a personal belief, "
110     "not as knowledge of future market prices."
111 ),
112 }

```

A.3 Full question and private-history template

The template includes the entire available private history and public tape. The runtime resolves it before elicitation. It explicitly states the horizon, interest/dividend ordering, and terminal redemption.

```

115 def question_text(period, periods):
116     return (
117         f"Period {period} of {periods}; {periods - period + 1} trading periods remain, including
118         this one.\n\n"
119         "MARKET RULES\n"
120         "There are 12 traders. Each began with $100 cash and 4 shares. Submit one "
121         "sealed limit order: buy, sell, or hold. A buy limit is the most you will "
122         "pay per share; a sell limit is the least you will accept. The exchange "
123         "matches crossing orders and all filled orders receive one clearing price. "
124         "Unmatched orders expire. You cannot borrow or short shares. Buy quantities "
125         "are capped by cash divided by your limit; sells by your inventory. Quotes "
126         "round to cents. After trading, cash earns 5% interest, then each held share "
127         "pays the same dividend: $0.40 or $1.00, equally likely. "
128         f"After period {periods}'s income, each remaining share is redeemed for $14; you keep all
129         cash.\n\n"
130         "YOUR ACCOUNT (all money below is in dollars)\n"
131         "Cash: {{ shared_state.market.your_account.cash_cents / 100 }}. "
132         "Shares: {{ shared_state.market.your_account.shares }}.\n"
133         "Your previous decisions and fills:\n"
134         "{% for h in shared_state.market.your_account.history %}"
135         "Period {{ h.period }}: {{ h.side }} {{ h.decision.quantity }} at limit "
136         "{{ h.limit_cents / 100 }}; admitted {{ h.accepted_quantity }}; signed fill "
137         "{{ h.fill }}; market price {{ h.transaction_price }}; interest "
138         "{{ h.interest }}; dividend per share {{ h.dividend_per_share }}. "
139         "Your forecasts (now, +2, +5, +10): {{ h.decision.forecast_0 }}, "
140         "{{ h.decision.forecast_2 }}, {{ h.decision.forecast_5 }}, "
141         "{{ h.decision.forecast_10 }}. Your rationale: {{ h.decision.rationale }}\n"
142         "{% else %}No previous decisions.\n{% endfor %}\n"

```

```

141     "PUBLIC HISTORY\n"
142     "{% for row in shared_state.market.tape %}"
143     "Period {{ row.period }}: transaction price {{ row.price }}, volume "
144     "{{ row.volume }}, dividend per share {{ row.dividend }}, total cash "
145     "{{ row.total_cash }}, total shares {{ row.total_shares }}, total interest "
146     "{{ row.total_interest }}.\n"
147     "{% else %}The market has not traded yet. There is no opening reference quote.\n{% endfor %}"
148     "A null or None price means there was no transaction that period.\n\n"
149     "YOUR DECISION\n"
150     "Forecast transaction prices now and 2, 5, and 10 periods ahead, then choose "
151     "your order using your trading approach. For forecast horizons beyond the "
152     "last trading period, use the terminal redemption amount. Return forecast_0, "
153     "forecast_2, forecast_5, forecast_10, side ('buy', 'sell', or 'hold'), price, "
154     "integer quantity, and a brief rationale. All forecasts and prices are in "
155     "dollars. For hold use price=0 and quantity=0. Retain any precommitted "
156     "schedule or exit plan in your rationale.\n"
157 )

```

A.4 Agents and native economic rules

The `call_market` factory supplies the standard admission and settlement commands. Income order and the marginal-midpoint price rule are explicit.

```

160 def build_agents():
161     """Two traders per archetype, with a reproducible seat assignment."""
162     types = list(COMPOSITION)
163     random.Random(SEED).shuffle(types)
164     agents = [
165         Agent(
166             name=f"trader-{seat:02d}",
167             traits={"role": "trader", "seat": seat},
168             instruction=COMMON + PERSONAS[kind],
169         )
170         for seat, kind in enumerate(types)
171     ]
172     agents.append(Agent(name="exchange", traits={"role": "exchange"}))
173     return agents
174
175
176 def build_market(trader_names):
177     """Declare economics; standard EDSL commands admit and settle the orders."""
178     return call_market(
179         trader_names,
180         periods=PERIODS,
181         seed=SEED,
182         initial_cash_cents=10000,
183         initial_shares=4,
184         initial_reference_price=14.0, # Internal only; never shown as a quote.
185         redemption_cents=1400,
186         pricing_rule="marginal_midpoint",
187         maximum_buy_quantity=48,
188         income_schedule=[
189             {"kind": "cash_interest", "rate": 0.05},
190             {
191                 "kind": "share_dividend",
192                 "values_cents": [40, 100],

```

```

193         "probabilities": [0.5, 0.5],
194         "sampling": "python_random_choice_v1",
195     },
196 ],
197     forecast_keys=KEYS[:4],
198 )

```

A.5 The EDSL execution plan

The exchange receives a literal scripted answer. That answer triggers the settlement state command; no model performs the accounting.

```

201 def build_execution_plan():
202     """EDSL asks traders; the exchange receives a literal scripted answer."""
203     return (
204         ExecutionPlan()
205         .bind(
206             role("trader"),
207             llm(
208                 model="gpt-5-mini",
209                 service="openai",
210                 parameters={
211                     "temperature": 1,
212                     "reasoning_effort": "medium",
213                     "max_tokens": 8000,
214                 },
215             ),
216         )
217         .bind(role("exchange"), scripted(answers={"settlement": "clear"}))
218     )

```

A.6 All thirty rounds and the serializable bundle

Every order step reads the session market and writes one order. Its settlement step depends on all assigned traders. The next round depends on settlement. There are no observation pause predicates in this construction.

```

221 def build(*, created_at=None):
222     """Build a serializable experiment: state, participants, workflow, execution."""
223     agents = build_agents()
224     machine = build_market([a.name for a in agents if a.traits["role"] == "trader"])
225     states = SharedStateMap(
226         SharedState(market=machine), state_id="portable-asset-market"
227     )
228     market = states.by("session").market
229     workflow = Workflow(
230         "Speculative asset market: full 30-round session",
231         metadata={
232             "economic_horizon": PERIODS,
233             "prompt_variant": "speculative-v1",
234             "observation_design": "Full 30 rounds fixed before execution; no early observation
235             pauses.",
236         },
237     )

```

```

237 previous = None
238 for period in range(1, PERIODS + 1):
239     question = QuestionDict(
240         question_name="decision",
241         question_text=question_text(period, PERIODS),
242         answer_keys=KEYS,
243         value_types=["float"] * 4 + ["str", "float", "int", "str"],
244         include_comment=False,
245     )
246     orders = workflow.step(
247         f"orders-{period}",
248         Survey([question]),
249         assigned_to=role("trader"),
250         after=previous,
251         visible_to=role("exchange"),
252         reads=(market.read(),),
253         writes=(
254             market.submit(
255                 trader=current.agent.name, period=period, decision=question.answer
256             ),
257         ),
258         metadata={"period": period, "phase": "sealed orders"},
259     )
260     previous = workflow.step(
261         f"settle-{period}",
262         Survey(
263             [
264                 QuestionFreeText(
265                     question_name="settlement",
266                     question_text="Run deterministic market settlement.",
267                 )
268             ]
269         ),
270         assigned_to=role("exchange"),
271         after=orders,
272         writes=(market.settle(period=period),),
273         metadata={"period": period},
274     )
275 return WorkflowExperiment(
276     workflow.compile(),
277     [states],
278     agents,
279     build_execution_plan(),
280     metadata={
281         "seed": SEED,
282         "prompt_variant": "speculative-v1",
283         "economic_horizon": PERIODS,
284         "observation_horizon": PERIODS,
285         "fresh_model_calls": True,
286         "created_at": created_at or datetime.now(timezone.utc).isoformat(),
287     },
288 )

```

A.7 Full command-line entry point

Supplying `-responses` replays saved answers and prohibits fallback to model calls. Omitting it launches paid hosted inference. Preparation alone writes the definition without executing it. Resume reloads the pinned JSON.

```

291 def main():
292     parser = argparse.ArgumentParser(description=__doc__)
293     parser.add_argument("--output", type=Path, required=True)
294     parser.add_argument(
295         "--responses", type=Path, help="Saved answers; prevents model calls"
296     )
297     parser.add_argument("--resume", action="store_true")
298     parser.add_argument("--prepare-only", action="store_true")
299     args = parser.parse_args()
300     specification = args.output / "specification.json"
301     if args.prepare_only:
302         args.output.mkdir(parents=True, exist_ok=True)
303         if specification.exists():
304             raise FileExistsError(specification)
305         build().save(specification)
306         print(json.dumps({"status": "prepared", "path": str(specification)}))
307         return
308     if args.resume:
309         experiment = WorkflowExperiment.load(args.output / "experiment.json")
310     elif specification.exists():
311         experiment = WorkflowExperiment.load(specification)
312     else:
313         experiment = build()
314     responses = None
315     if args.responses:
316         responses = json.loads(args.responses.read_text())
317     else:
318         # Hosted execution uses locally configured provider credentials.
319         from dotenv import load_dotenv
320
321         load_dotenv()
322     result = experiment.run(args.output, responses=responses, resume=args.resume)
323     (args.output / "completion.json").write_text(json.dumps(result, indent=2) + "\n")
324     from edsl.sharedstate import SQLiteStateBackend
325
326     state = (
327         SQLiteStateBackend(experiment.states[0], args.output / "state-0.sqlite")
328         .snapshot("session")
329         .state["market"]
330     )
331     (args.output / "market.json").write_text(json.dumps(state, indent=2) + "\n")
332     assert result["status"] == "completed" and result["completed_items"] == 390
333     assert state["finished"] and len(state["tape"]) == PERIODS
334     assert len(state["order_log"]) == 12 * PERIODS
335     assert all(a["shares"] == 0 for a in state["accounts"].values())
336     print(json.dumps({"status": "ok", "data": result, "warnings": []}))
337
338
339 if __name__ == "__main__":
340     main()

```

B Complete native workflow executor

The following is the complete `edsl/workflows/experiment.py` source used for the batched continuation, copied to `code/edsl/workflow_experiment.py`. It shows serialization and validation, durable launch/recovery, and the actual EDSL job construction and execution. This is the updated implementation; the initial serial implementation is identified by its hash in the original source manifest. It is not silently represented as the batched version.

B.1 Bundle, validation, and save/load

The object contains workflow, states, agents, execution plan, and metadata. Python callbacks and functional questions are rejected by this portable API; standard capability implementations must be present in the installed runtime.

```

1  """Portable workflow, state, participant, and executor specifications.
2
3  Only data is loaded. Algorithm implementations must already be available in
4  the selected EDSL runtime; no source text, imports, or pickle is executed.
5  """
6
7  from copy import deepcopy
8  import json
9  from pathlib import Path
10
11 from edsl import Agent, Model, Scenario, Survey
12 from edsl._data_contracts import definition_fingerprint, validate_data
13 from edsl.sharedstate import SharedStateMap, SQLiteStateBackend
14 from edsl.sharedstate.dsl_runtime import default_runtime
15
16 from .coordinator import WorkflowCoordinator
17 from .definition import HumanWorkflow
18 from .execution import ExecutionPlan
19 from .store import SQLiteWorkflowStore
20
21
22 class WorkflowExperiment:
23     """A JSON-portable experiment with explicit runtime capability dependencies."""
24
25     def __init__(
26         self, workflow, states, agents, execution_plan, *, metadata=None, runtime=None
27     ):
28         agents = tuple(agents)
29         if any(
30             a.has_dynamic_traits_function or hasattr(a, "answer_question_directly")
31             for a in agents
32         ):
33             raise ValueError("portable agents cannot contain Python callbacks")
34         self._check_questions(workflow.to_dict())
35         self.workflow = HumanWorkflow.from_dict(workflow.to_dict())
36         self.states = tuple(SharedStateMap.from_dict(s.to_dict()) for s in states)
37         self.agents = tuple(Agent.from_dict(a.to_dict()) for a in agents)
38         self.execution_plan = ExecutionPlan.from_dict(execution_plan.to_dict())
39         self.metadata = deepcopy(metadata or {})
40         self.runtime = runtime or default_runtime()
41         self.validate()
42
43     def validate(self):

```

```

44     validate_data(self.to_dict(), path="workflow experiment")
45     maps = {s.state_id: s for s in self.states}
46     if len(maps) != len(self.states):
47         raise ValueError("experiment state IDs must be unique")
48     names = [a.name for a in self.agents]
49     if any(not n for n in names) or len(names) != len(set(names)):
50         raise ValueError("experiment agents require unique names")
51     for state in self.states:
52         for machine in state.definition.machines.values():
53             self.runtime.validate_capabilities(machine)
54     operations = [
55         op for step in self.workflow.steps for op in (*step.reads, *step.writes)
56     ]
57     operations += [rule.read for rule in self.workflow.pause_rules]
58     for op in operations:
59         if op.state_id not in maps or definition_fingerprint(
60             op.definition.to_dict()
61         ) != definition_fingerprint(maps[op.state_id].definition.to_dict()):
62             raise ValueError(
63                 "workflow references an absent or different state definition"
64             )
65     for step in self.workflow.steps:
66         if not any(step.assignee.matches(a.traits) for a in self.agents):
67             raise ValueError(f"step {step.name!r} has no assigned participant")
68     for agent in self.agents:
69         executor = self.execution_plan.resolve(agent.traits)
70         if executor.kind == "llm" and not executor.options.get("model"):
71             raise ValueError("LLM executor must name a model")
72         if executor.kind == "scripted" and not isinstance(
73             executor.options.get("answers"), dict
74         ):
75             raise ValueError("portable scripted executor requires literal answers")
76
77     def to_dict(self):
78         return {
79             "type": "workflow_experiment",
80             "version": 1,
81             "workflow": self.workflow.to_dict(),
82             "states": [s.to_dict() for s in self.states],
83             "agents": [a.to_dict() for a in self.agents],
84             "execution_plan": self.execution_plan.to_dict(),
85             "metadata": deepcopy(self.metadata),
86         }
87
88     @classmethod
89     def from_dict(cls, data, *, runtime=None):
90         if data.get("type") != "workflow_experiment" or data.get("version") != 1:
91             raise ValueError("unsupported workflow experiment serialization")
92         cls._check_questions(data["workflow"])
93         if any("traits" not in agent for agent in data["agents"]):
94             raise ValueError(
95                 "portable agents require the data-only traits representation"
96             )
97         return cls(
98             HumanWorkflow.from_dict(data["workflow"]),
99             [SharedStateMap.from_dict(s) for s in data["states"]],
100             [Agent.from_dict(deepcopy(a)) for a in data["agents"]],
101             ExecutionPlan.from_dict(data["execution_plan"]),
102             metadata=data.get("metadata"),
103             runtime=runtime,

```

```

104         )
105
106     @staticmethod
107     def _check_questions(workflow):
108         for step in workflow["steps"]:
109             for question in step["survey"]["questions"]:
110                 if question.get("question_type") == "functional":
111                     raise ValueError(
112                         "portable experiments do not embed functional Python questions"
113                     )
114
115     def save(self, path):
116         path = Path(path)
117         temporary = path.with_name(path.name + ".tmp")
118         temporary.write_text(
119             json.dumps(self.to_dict(), indent=2, allow_nan=False) + "\n"
120         )
121         temporary.replace(path)
122
123     @classmethod
124     def load(cls, path, *, runtime=None):
125         return cls.from_dict(json.loads(Path(path).read_text()), runtime=runtime)

```

B.2 Durable execution and recovery

Work-item views are opened before answering any member of the ready batch. Compatible LLM items are grouped by workflow step, model settings, and survey schema. Scripted items and supplied replay responses bypass inference jobs.

```

127     def run(self, output, *, responses=None, resume=False, instance_id="experiment"):
128         """Execute until a declared pause or completion.
129
130         ‘‘responses‘‘ is optional replay data: a sequence of dictionaries with
131         ‘‘step‘‘, ‘‘participant‘‘, and ‘‘answers‘‘. Supplying it forbids model
132         calls, including when an answer is missing. Without it, configured LLM
133         executors call their hosted provider through EDSL. All questions in a
134         ready batch are opened before submitting any of that batch’s answers.
135         Live LLM work in each ready batch is executed as one EDSL Jobs object,
136         preserving each interview’s own rendered survey, agent, and model.
137         """
138         output = Path(output)
139         pinned = output / "experiment.json"
140         if pinned.exists():
141             if not resume:
142                 raise FileExistsError("saved experiment exists; use resume=True")
143             if json.loads(pinned.read_text()) != self.to_dict():
144                 raise ValueError("saved experiment specification changed")
145         elif resume:
146             raise FileNotFoundError("no experiment to resume")
147         replay = None
148         if responses is not None:
149             replay = {}
150             for row in responses:
151                 key = (row["step"], row["participant"])
152                 if key in replay:
153                     raise ValueError(f"duplicate replay response: {key}")
154                 replay[key] = row["answers"]

```

```

155     self.validate()
156     output.mkdir(parents=True, exist_ok=True)
157     if not pinned.exists():
158         self.save(pinned)
159     store = SQLiteWorkflowStore(output / "workflow.sqlite")
160     backends = {
161         s.state_id: SQLiteStateBackend(
162             s, output / f"state-{i}.sqlite", runtime=self.runtime
163         )
164         for i, s in enumerate(self.states)
165     }
166     if resume:
167         coordinator = WorkflowCoordinator.restore(
168             instance_id, store, state_backends=backends
169         )
170         coordinator.recover(instance_id, max_attempts=3)
171         if store.instance_status(instance_id) == "paused":
172             coordinator.resume(instance_id)
173     else:
174         coordinator = WorkflowCoordinator(
175             self.workflow, store, state_backends=backends
176         )
177         coordinator.launch(
178             self.agents,
179             instance_id=instance_id,
180             random_seed=self.metadata.get("seed", instance_id),
181         )
182     agents = {a.name: a for a in self.agents}
183     while pending := store.pending_outbox(instance_id):
184         opened_items = []
185         for row in pending:
186             store.mark_delivered(row["id"])
187             opened_items.append(coordinator.open(row["work_item_id"]))
188         if replay is None:
189             llm_items = [
190                 item
191                 for item in opened_items
192                 if self.execution_plan.resolve(
193                     agents[item.participant_id].traits
194                 ).kind
195                 == "llm"
196             ]
197             groups = {}
198             for item in llm_items:
199                 options = self.execution_plan.resolve(
200                     agents[item.participant_id].traits
201                 ).options
202                 key = (
203                     item.step_name,
204                     json.dumps(options, sort_keys=True),
205                     json.dumps(self._batch_survey(item).to_dict(), sort_keys=True),
206                 )
207                 groups.setdefault(key, []).append(item)
208             for group in groups.values():
209                 self._run_llm_batch(group, agents, coordinator, store, output)
210             opened_items = [item for item in opened_items if item not in llm_items]
211         for opened in opened_items:
212             # Scripted work and supplied replay answers need no inference job.
213             attempt = store.start_attempt(opened.id, lease_seconds=600)
214             agent = agents[opened.participant_id]

```

```

215         executor = self.execution_plan.resolve(agent.traits)
216         store.record_executor(opened.id, executor.kind, executor.options)
217         try:
218             if executor.kind == "scripted":
219                 answers = deepcopy(executor.options["answers"])
220             elif replay is not None:
221                 answers = deepcopy(replay[(opened.step_name, agent.name)])
222             else:
223                 raise ValueError(
224                     "human work needs a human delivery adapter or replay responses"
225                 )
226             coordinator.submit(
227                 opened.id,
228                 answers,
229                 idempotency_key=f"experiment:{opened.id}",
230                 attempt_id=attempt["id"],
231             )
232         except Exception as exc:
233             if store.item(opened.id)["status"] != "committing":
234                 store.finish_attempt(
235                     attempt["id"],
236                     status="failed",
237                     error_kind="exception",
238                     error_message=str(exc)[:1000],
239                 )
240                 store.retry_item(
241                     opened.id, reason="resume after executor failure"
242                 )
243             raise
244         status = store.instance_status(instance_id)
245         if status not in {"paused", "completed"}:
246             raise RuntimeError("workflow has unfinished work without a declared pause")
247         return {
248             "status": status,
249             "instance_id": instance_id,
250             "completed_items": sum(
251                 i["status"] == "completed" for i in store.items(instance_id)
252             ),
253             "pauses": [
254                 e for e in store.events(instance_id) if e["kind"] == "workflow.paused"
255             ],
256         }

```

B.3 One paired EDSL job per compatible batch

Each participant gets its own scenario containing frozen question text and a work-item ID. `zip_assign(over=("agents", "scenarios"))` pairs row i with row i , producing 12 interviews rather than a 144-interview cross-product. Private text is not put into a scenario shared by all traders. Responses are matched by ID and checked against the expected participant, independently of completion order. Accepted submissions are retained if another result fails; unfinished work is made retryable. Idempotency protects committed state changes, not a blanket guarantee against repeated provider billing after a network failure or crash.

```

258     @staticmethod
259     def _batch_survey(item):
260         """Parameterize text only; differing response schemas get separate jobs."""

```

```

261     data = item.survey.to_dict()
262     for question in data["questions"]:
263         if "question_name" in question:
264             name = question["question_name"]
265             question["question_text"] = (
266                 "{ workflow_question_texts[" + repr(name) + " ]}"
267             )
268     data["memory_plan"]["survey_question_texts"] = [
269         q["question_text"] for q in data["questions"] if "question_name" in q
270     ]
271     return Survey.from_dict(data)
272
273 def _run_llm_batch(self, opened_items, agents, coordinator, store, output):
274     """One EDSL job per compatible step; match unordered results by participant."""
275     from datetime import datetime, timezone
276     import time
277
278     attempts, participants, scenarios = {}, [], []
279     by_id = {item.id: item for item in opened_items}
280     started = time.monotonic()
281     try:
282         for item in opened_items:
283             attempts[item.id] = store.start_attempt(item.id, lease_seconds=600)
284             agent = agents[item.participant_id]
285             executor = self.execution_plan.resolve(agent.traits)
286             options = executor.options
287             store.record_executor(item.id, executor.kind, options)
288             model = Model(
289                 options["model"],
290                 service_name=options.get("service"),
291                 **options.get("parameters", {}),
292             )
293             store.record_model(item.id, model.to_dict())
294             participants.append(agent)
295             scenarios.append(
296                 Scenario(
297                     {
298                         "workflow_work_item_id": item.id,
299                         "workflow_question_texts": {
300                             q.question_name: q.question_text
301                             for q in item.survey.questions
302                         },
303                     },
304                 )
305             )
306         if len({a.name for a in participants}) != len(opened_items):
307             raise ValueError("batch requires distinct participants")
308         # Pair agent i only with its own frozen private scenario i.
309         job = (
310             self._batch_survey(opened_items[0])
311             .by(participants)
312             .by(scenarios)
313             .by(model)
314             .zip_assign(over=("agents", "scenarios"))
315         )
316         jobs_folder = output / "jobs"
317         jobs_folder.mkdir(exist_ok=True)
318         job_path = (
319             jobs_folder
320             / f"batch-{{opened_items[0].id}}-{{attempts[opened_items[0].id]['id']}}.json"

```

```

321     )
322     job_path.write_text(json.dumps(job.to_dict(), indent=2) + "\n")
323     batch = {
324         "started_at": datetime.now(timezone.utc).isoformat(),
325         "work_item_ids": list(by_id),
326         "steps": sorted({item.step_name for item in opened_items}),
327         "interviews": len(opened_items),
328         "job_path": str(job_path.relative_to(output)),
329     }
330     with (output / "inference-batches.jsonl").open("a") as handle:
331         handle.write(json.dumps(**batch, "event": "started") + "\n")
332     results = job.run(
333         disable_remote_inference=True,
334         disable_remote_cache=True,
335         cache=False,
336         stop_on_exceptions=False,
337     )
338     seen, failures = set(), []
339     for result in results:
340         item_id = result.scenario["workflow_work_item_id"]
341         if item_id not in by_id or item_id in seen:
342             raise ValueError("unexpected or duplicate work-item result")
343         seen.add(item_id)
344         item = by_id[item_id]
345         if result.agent.name != item.participant_id:
346             raise ValueError("result participant differs from work item")
347         record = {
348             "work_item_id": item_id,
349             "step": item.step_name,
350             "participant": item.participant_id,
351             "result": result.to_dict(),
352         }
353         with (output / "model-calls.jsonl").open("a") as handle:
354             handle.write(json.dumps(record, allow_nan=False) + "\n")
355         try:
356             required = {q.question_name for q in item.survey.questions}
357             if not required.issubset(result.answer):
358                 raise ValueError("incomplete model result")
359             coordinator.submit(
360                 item_id,
361                 dict(result.answer),
362                 idempotency_key=f"experiment:{item_id}",
363                 attempt_id=attempts[item_id]["id"],
364             )
365         except Exception as exc:
366             failures.append(exc)
367     if seen != set(by_id):
368         failures.append(ValueError("EDSL job returned an incomplete batch"))
369     if failures:
370         raise failures[0]
371     with (output / "inference-batches.jsonl").open("a") as handle:
372         handle.write(
373             json.dumps(
374                 {
375                     **batch,
376                     "event": "completed",
377                     "results": len(results),
378                     "elapsed_seconds": time.monotonic() - started,
379                 }
380             )

```

```

381         + "\n"
382     )
383     except Exception as exc:
384         # Keep successful submissions; only unfinished interviews retry.
385         for item_id, attempt in attempts.items():
386             if store.item(item_id)["status"] not in {"completed", "committing"}:
387                 store.finish_attempt(
388                     attempt["id"],
389                     status="failed",
390                     error_kind="exception",
391                     error_message=str(exc)[:1000],
392                 )
393                 store.retry_item(item_id, reason="resume after EDSL batch failure")
394         raise

```

C Complete native call-market implementation

The source below is `edsl/sharedstate/call_market.py`, copied without abbreviation. It defines the serialized rule parameters, private read view, versioned commands, admission, and settlement. This is library code invoked by the authoring file; it is not embedded Python inside the saved experiment JSON.

C.1 Supported rules, machine construction, and private view

The runtime validates policy choices before creating an execution database. The view returns the requesting trader's account and public history, not the full accounts map or the sealed order book. Unsupported versions or policies fail explicitly.

```

1  """Version-1 sealed call market: explicit monetary and clearing rules.
2
3  The serialized Machine carries all economic parameters. Implementations are
4  standard runtime capabilities; loading never imports experiment source code.
5  Amounts in state and rule parameters are integer cents, except quoted prices
6  and elicited forecasts, which are in dollars. Random streams are specified by
7  seed and purpose; Python's versioned choice/shuffle behavior is part of v1.
8  """
9
10 from copy import deepcopy
11 from decimal import Decimal, ROUND_HALF_UP
12 import math
13 import random
14
15 from .dsl import Command, Machine, T, algorithm, current, field, input_, state_field
16
17
18 def validate_rules(constants):
19     """Reject unsupported or ambiguous policies before a backend starts work."""
20
21     def integer(name, minimum):
22         value = constants[name]
23         if isinstance(value, bool) or not isinstance(value, int) or value < minimum:
24             raise ValueError(f"call market {name} must be an integer >= {minimum}")
25
26     for name, minimum in [

```

```

27     ("periods", 1),
28     ("total_shares", 0),
29     ("maximum_buy_quantity", 1),
30     ("redemption_cents", 0),
31 ]:
32     integer(name, minimum)
33 if isinstance(constants["seed"], bool) or not isinstance(
34     constants["seed"], (str, int)
35 ):
36     raise ValueError("call market seed must be a string or integer")
37 fixed = {
38     "rounding": "half_up",
39     "money_decimals": 2,
40     "tie_breaking": "seeded_price_priority_v1",
41     "borrowing": False,
42     "short_selling": False,
43     "unfilled_orders": "expire",
44     "redemption_timing": "after_final_income",
45 }
46 for key, value in fixed.items():
47     if constants.get(key) != value:
48         raise ValueError(f"unsupported call market {key}: expected {value!r}")
49 if constants["pricing_rule"] not in {
50     "marginal_midpoint",
51     "marginal_bid",
52     "marginal_ask",
53 }:
54     raise ValueError("unsupported call market pricing_rule")
55 keys = constants["forecast_keys"]
56 if (
57     not isinstance(keys, (list, tuple))
58     or any(not isinstance(k, str) or not k for k in keys)
59     or len(keys) != len(set(keys))
60 ):
61     raise ValueError("forecast_keys must be distinct nonempty strings")
62 schedule = constants["income_schedule"]
63 if (
64     not isinstance(schedule, (list, tuple))
65     or len(schedule) != 2
66     or sorted(r["kind"] for r in schedule) != ["cash_interest", "share_dividend"]
67 ):
68     raise ValueError(
69         "income_schedule must order one interest and one dividend payment"
70     )
71 for rule in schedule:
72     if rule["kind"] == "cash_interest":
73         rate = rule["rate"]
74         if (
75             isinstance(rate, bool)
76             or not isinstance(rate, (int, float))
77             or not math.isfinite(rate)
78             or rate < 0
79         ):
80             raise ValueError("interest rate must be finite and nonnegative")
81     else:
82         values, probabilities = rule["values_cents"], rule["probabilities"]
83         if not values or any(
84             isinstance(v, bool) or not isinstance(v, int) or v < 0 for v in values
85         ):
86             raise ValueError("dividends must be nonnegative integer cents")

```

```

87         if (
88             len(values) != len(probabilities)
89             or any(
90                 isinstance(p, bool)
91                 or not isinstance(p, (int, float))
92                 or not math.isfinite(p)
93                 or p < 0
94                 for p in probabilities
95             )
96             or not math.isclose(sum(probabilities), 1)
97         ):
98             raise ValueError("dividend probabilities must sum to one")
99         if rule["sampling"] == "python_random_choice_v1":
100             if any(not math.isclose(p, 1 / len(values)) for p in probabilities):
101                 raise ValueError("choice sampling requires equal probabilities")
102             elif rule["sampling"] != "python_random_choices_v1":
103                 raise ValueError("unsupported dividend sampling rule")
104
105
106 def call_market(
107     traders,
108     *,
109     periods=30,
110     seed=1,
111     initial_cash_cents=10000,
112     initial_shares=4,
113     initial_reference_price=14.0,
114     redemption_cents=1400,
115     pricing_rule="marginal_midpoint",
116     maximum_buy_quantity=None,
117     income_schedule=None,
118     forecast_keys=("forecast_0", "forecast_2", "forecast_5", "forecast_10"),
119 ):
120     """Build a portable call-market Machine using standard versioned operations.
121
122     ‘‘income_schedule‘‘ is ordered. Each round trades first, applies this
123     schedule, then redeems shares if it is the final economic period.
124     Observation pauses do not change ‘‘periods‘‘ or redeem inventory.
125     """
126     traders = list(traders)
127     if (
128         not traders
129         or any(not isinstance(t, str) or not t for t in traders)
130         or len(traders) != len(set(traders))
131     ):
132         raise ValueError("traders must be distinct nonempty names")
133     for value in [initial_cash_cents, initial_shares]:
134         if isinstance(value, bool) or not isinstance(value, int) or value < 0:
135             raise ValueError("initial cash and shares must be nonnegative integers")
136     constants = dict(
137         periods=periods,
138         seed=seed,
139         total_shares=initial_shares * len(traders),
140         maximum_buy_quantity=(
141             max(1, initial_shares * len(traders))
142             if maximum_buy_quantity is None
143             else maximum_buy_quantity
144         ),
145         redemption_cents=redemption_cents,
146         pricing_rule=pricing_rule,

```

```

147     rounding="half_up",
148     money_decimals=2,
149     borrowing=False,
150     short_selling=False,
151     tie_breaking="seeded_price_priority_v1",
152     unfilled_orders="expire",
153     redemption_timing="after_final_income",
154     forecast_keys=list(forecast_keys),
155     income_schedule=deepcopy(
156         income_schedule
157         if income_schedule is not None
158         else [
159             {"kind": "cash_interest", "rate": 0.05},
160             {
161                 "kind": "share_dividend",
162                 "values_cents": [40, 100],
163                 "probabilities": [0.5, 0.5],
164                 "sampling": "python_random_choice_v1",
165             },
166         ]
167     ),
168 )
169 validate_rules(constants)
170 machine = Machine(
171     name="CallMarket",
172     constants=constants,
173     fields={
174         "accounts": state_field(
175             T.map(),
176             {
177                 t: {
178                     "cash_cents": initial_cash_cents,
179                     "shares": initial_shares,
180                     "history": [],
181                 }
182                 for t in traders
183             },
184         ),
185         "orders": state_field(T.map(), {}),
186         "order_log": state_field(T.sequence(), []),
187         "tape": state_field(T.sequence(), []),
188         "period": state_field(T.integer(), 1),
189         "last_price": state_field(T.optional(T.number()), initial_reference_price),
190         "finished": state_field(T.boolean(), False),
191     },
192     commands={
193         "submit": Command(
194             inputs={"trader": T.text(), "period": T.integer(), "decision": T.map()},
195             effects=(
196                 algorithm(
197                     "call_market_submit",
198                     trader=input_("trader"),
199                     period=input_("period"),
200                     decision=input_("decision"),
201                 ),
202             ),
203         ),
204         "settle": Command(
205             inputs={"period": T.integer()},
206             effects=(algorithm("call_market_settle", period=input_("period")),),

```

```

207     ),
208     },
209     view={
210         "period": field("period"),
211         "tape": field("tape"),
212         "last_price": field("last_price"),
213         "your_account": field("accounts").get(current("name")),
214         "finished": field("finished"),
215     },
216     algorithms=("call_market_submit@1", "call_market_settle@1"),
217 )
218 machine.validate()
219 return machine

```

C.2 Integer-cent conversion and sealed order admission

Forecasts must be finite and nonnegative. This validation does not assess valuation reasoning or enforce terminal-horizon forecast compliance.

```

222 def cents(value):
223     return int(
224         (Decimal(str(value)) * 100).quantize(Decimal("1"), rounding=ROUND_HALF_UP)
225     )
226
227
228 def submit_order(state, inputs, constants):
229     """Commit a sealed order without changing balances or the public tape."""
230     trader, period, answer = inputs["trader"], inputs["period"], inputs["decision"]
231     if (
232         period != state["period"]
233         or state["finished"]
234         or trader not in state["accounts"]
235     ):
236         raise ValueError("Invalid trader, stale period, or finished market")
237     if trader in state["orders"]:
238         raise ValueError("Trader already submitted this period")
239     account = state["accounts"][trader]
240     error = None
241     for key in constants["forecast_keys"]:
242         val = answer.get(key)
243         if (
244             isinstance(val, bool)
245             or not isinstance(val, (int, float))
246             or not math.isfinite(val)
247             or val < 0
248         ):
249             raise ValueError(f"{key} must be a finite nonnegative number")
250     side, qty, price = answer.get("side"), answer.get("quantity"), answer.get("price")
251     if side not in {"buy", "sell", "hold"}:
252         error = "invalid side"
253     elif isinstance(qty, bool) or not isinstance(qty, int) or qty < 0:
254         error = "quantity must be a nonnegative integer"
255     elif (
256         isinstance(price, bool)
257         or not isinstance(price, (int, float))
258         or not math.isfinite(price)
259         or price < 0

```

```

260 ):
261     error = "price must be finite and nonnegative"
262     elif side != "hold" and (cents(price) <= 0 or qty == 0):
263         error = "active orders require positive price and quantity"
264     limit = cents(price) if error is None else 0
265     accepted_qty = 0
266     if error is None and side == "buy":
267         accepted_qty = min(
268             qty, account["cash_cents"] // limit, constants["maximum_buy_quantity"]
269         )
270     elif error is None and side == "sell":
271         accepted_qty = min(qty, account["shares"])
272     state["orders"][trader] = {
273         "trader": trader,
274         "period": period,
275         "decision": dict(answer),
276         "side": side,
277         "limit_cents": limit,
278         "accepted_quantity": accepted_qty,
279         "rejection": error,
280     }

```

C.3 Full clearing, income, and redemption logic

The tie lottery and common dividend stream use distinct seed strings. Settlement checks every trader has submitted, conserves cash and shares under trading, applies the ordered income schedule, records fills, and redeems shares only at the final round.

```

283 def settle_market(state, inputs, constants):
284     """Maximal crossing volume, configured uniform price, then scheduled income."""
285     period = inputs["period"]
286     if period != state["period"] or state["finished"]:
287         raise ValueError("Settlement must occur exactly once per current period")
288     if set(state["orders"]) != set(state["accounts"]):
289         raise ValueError("Cannot clear until every trader has submitted")
290     # A reproducible per-period lottery breaks equal-price ties independently of
291     # completion order. Admission bounds the quantities expanded into unit orders.
292     names = sorted(state["accounts"])
293     random.Random(f"{constants['seed']}:{period}").shuffle(names)
294     priority = {name: i for i, name in enumerate(names)}
295     bids, asks = [], []
296     for name, order in state["orders"].items():
297         target = bids if order["side"] == "buy" else asks
298         target.extend(
299             (order["limit_cents"], priority[name], name)
300             for _ in range(order["accepted_quantity"])
301         )
302     bids.sort(key=lambda row: (-row[0], row[1]))
303     asks.sort(key=lambda row: (row[0], row[1]))
304     volume = 0
305     for bid, ask in zip(bids, asks):
306         if bid[0] < ask[0]:
307             break
308         volume += 1
309     price = None
310     if volume:
311         bid, ask = bids[volume - 1][0], asks[volume - 1][0]

```

```

312     price = {
313         "marginal_midpoint": (bid + ask + 1) // 2,
314         "marginal_bid": bid,
315         "marginal_ask": ask,
316     }[constants["pricing_rule"]]
317 cash_before = sum(a["cash_cents"] for a in state["accounts"].values())
318 shares_before = sum(a["shares"] for a in state["accounts"].values())
319 fills = {name: 0 for name in names}
320 for bid, ask in zip(bids[:volume], asks[:volume]):
321     buyer, seller = state["accounts"][bid[2]], state["accounts"][ask[2]]
322     buyer["cash_cents"] -= price
323     seller["cash_cents"] += price
324     buyer["shares"] += 1
325     seller["shares"] -= 1
326     fills[bid[2]] += 1
327     fills[ask[2]] -= 1
328 assert sum(a["cash_cents"] for a in state["accounts"].values()) == cash_before
329 assert (
330     sum(a["shares"] for a in state["accounts"].values())
331     == shares_before
332     == constants["total_shares"]
333 )
334 dividend_rule = next(
335     r for r in constants["income_schedule"] if r["kind"] == "share_dividend"
336 )
337 rng = random.Random(f"{constants['seed']}:dividend:{period}")
338 values = dividend_rule["values_cents"]
339 dividend = (
340     rng.choice(values)
341     if dividend_rule["sampling"] == "python_random_choice_v1"
342     else rng.choices(values, weights=dividend_rule["probabilities"], k=1)[0]
343 )
344 total_interest = 0
345 for name, account in state["accounts"].items():
346     interest = 0
347     for rule in constants["income_schedule"]:
348         if rule["kind"] == "cash_interest":
349             interest = int(
350                 (
351                     Decimal(account["cash_cents"]) * Decimal(str(rule["rate"]))
352                 ).quantize(Decimal("1"), rounding=ROUND_HALF_UP)
353             )
354             account["cash_cents"] += interest
355             total_interest += interest
356         else:
357             account["cash_cents"] += dividend * account["shares"]
358     assert account["cash_cents"] >= 0 and account["shares"] >= 0
359     account["history"].append(
360         {
361             **state["orders"][name],
362             "fill": fills[name],
363             "transaction_price": None if price is None else price / 100,
364             "interest": interest / 100,
365             "dividend_per_share": dividend / 100,
366         }
367     )
368 if price is not None:
369     state["last_price"] = price / 100
370 row = {
371     "period": period,

```

```

372     "price": None if price is None else price / 100,
373     "reference_price": state["last_price"],
374     "volume": volume,
375     "dividend": dividend / 100,
376     "total_interest": total_interest / 100,
377     "total_cash": sum(a["cash_cents"] for a in state["accounts"].values()) / 100,
378     "total_shares": shares_before,
379 }
380 assert (
381     cents(row["total_cash"])
382     == cash_before + total_interest + dividend * shares_before
383 )
384 state["tape"].append(row)
385 state["order_log"].extend(state["orders"][name] for name in sorted(names))
386 state["orders"] = {}
387 if period == constants["periods"]:
388     for account in state["accounts"].values():
389         account["redeemed_shares"] = account["shares"]
390         account["cash_cents"] += constants["redemption_cents"] * account["shares"]
391         account["shares"] = 0
392     state["finished"] = True
393     state["period"] += 1

```

D Terminal accounts

Cash includes all realized income and terminal redemption. The final inventory of every trader is zero. “Redeemed” records shares held immediately before redemption. Difference is relative to passive final cash of \$648.69.

Trader	Type	Final cash (\$)	Difference (\$)	Redeemed
00	MC	281.09	-367.60	18
01	RA	742.05	+93.36	0
02	NT	777.05	+128.36	0
03	NT	828.00	+179.31	0
04	RA	742.05	+93.36	0
05	OC	817.14	+168.45	0
06	PC	648.69	+0.00	4
07	BT	631.12	-17.57	4
08	MC	462.07	-186.62	7
09	PC	788.07	+139.38	0
10	BT	811.86	+163.17	0
11	OC	255.15	-393.54	15

E Reproduction and artifact guide

E.1 Replay the exact saved answers without model calls

From the original repository root, replay through the complete consolidated source below. This market has no pause rules, so one invocation executes all 30 rounds. Use a new output directory.

```
python -m examples.asset_market.full_market_experiment \
  --responses examples/asset_market/full-market-note/data/responses.json \
  --output /private/tmp/full-market-source-replay-new
```

With only the report bundle, point `PYTHONPATH` at the updated EDSL checkout and run `code/full_market_experiment.py` with the equivalent paths. Installing an unrelated released EDSL version is insufficient. A new model session omits `-responses`; its answers and prices need not match this archive, even with the same seed.

E.2 What is included

Path	Contents
<code>report.pdf</code> , <code>report.tex</code> <code>code/</code>	Compiled report and editable LaTeX. Complete standalone experiment, audit/figure builders, native executor and market source, paired-assignment implementation, and historical builders.
<code>data/experiment.json</code>	Exact executed 30-round specification.
<code>data/responses.json</code>	All 360 answers for replay.
<code>data/market.json</code>	Final accounts, all histories, admitted orders, and tape.
<code>data/model-calls.jsonl.gz</code>	Compressed archive of all actual EDSL prompts and responses.
<code>data/round-13-job.json</code>	One complete saved 12-interview paired EDSL job.
<code>data/decisions.csv</code> , <code>data/periods.csv</code>	Decision/fill ledger and all 30 round outcomes.
<code>data/wealth-comparison.csv</code>	Final cash, passive benchmark, redeemed inventory.
<code>data/terminal-forecast-violations.csv</code>	All nine measured instruction violations.
<code>data/authoring-check.json</code> , <code>data/replay-check.json</code>	Specification comparison and complete saved-answer replay checks.
<code>data/batch-runner-transition.json</code>	Exact switchover record, PR revision, hashes, and test evidence.
<code>figures/</code> , <code>manifest.json</code>	Vector/PNG graphics and SHA-256 checksums.

The original SQLite databases and checkpoint backup remain in the repository’s run directory. The data in this bundle can reproduce the state through replay; it does not duplicate those databases. The source snapshots document the relevant code but are not a standalone EDSL installation. Recorded source hashes distinguish the serial and batched phases of the actual execution.

E.3 Regenerate figures and compile

The builder re-audits the saved run before copying artifacts or drawing figures. It makes no model calls:

```
python -m examples.asset_market.build_full_market_note
latexmk -cd -pdf -interaction=nonstopmode -halt-on-error \
  examples/asset_market/full-market-note/report.tex
```

A recipient of the bundle can compile `report.tex` directly: figures, relative-path source listings, and generated tables are already included.